CHAPTER 29

Stock Exchange

Match buy and sell orders with microsecond latency, strict fairness and zero data loss.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design an **electronic stock exchange** (NYSE / NASDAQ): participants (brokers) place **limit/market orders**; the exchange maintains an **order book** per symbol and **matches** buy and sell orders by price-time priority, publishes market data and executions. Requirements are extreme: **ultra-low latency, high availability, deterministic fairness and auditability**.

Why interviewers ask it: The most latency-sensitive design: single-threaded sequenced matching, event sourcing, shared-memory messaging, hot-standby failover.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Which securities?	Stocks only.
Order types?	Limit and market orders; cancel/replace; not options/futures.
Scale?	About 100 symbols and 1 B orders/day across the exchange.
Latency?	Tens of microseconds (tail latency also critical).
Availability?	99.99%; recover within seconds; no lost orders.
Users?	Brokers via FIX protocol; market-data subscribers.

3. Requirements

Functional

- Place a new order; cancel an order.
- Match buy/sell by price-time priority.
- Publish executions and market data (order-book levels, candlesticks).
- Risk checks before the order reaches the matching engine.

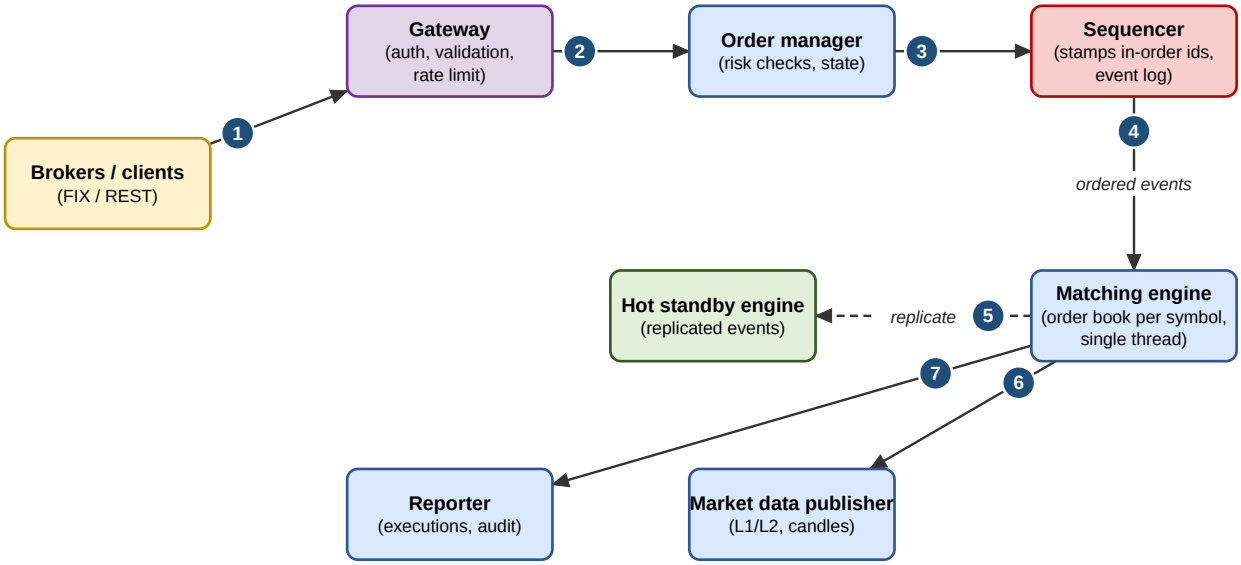
Non-functional

- **Availability** 99.99%.
- **Fault tolerance** with fast failover and recovery (no loss).
- **Latency** in microseconds (including 99th percentile).
- **Security:** DDoS protection, access control, audit.

4. Back-of-the-envelope estimation

- 100 symbols, 1 B orders/day, trading ~6.5 h/day = 23,400 s -> about **43 K orders/s average**; peak (open/close) 5x = about **215 K orders/s**.
- Per-symbol throughput: matching of one symbol is **strictly sequential**; a single thread can do hundreds of thousands to millions of ops/s in memory.

5. High-level design



Critical path = gateway -> order manager -> sequencer -> matching engine; everything else is off the hot path.

Step	What happens
1	A broker sends an order (`BUY 100 AAPL @ 150.00`) over FIX or REST to the **gateway**: authentication, validation, rate limits and encryption.
2	The **order manager** performs pre-trade risk checks (funds, position limits, circuit breakers) and tracks the order state.
3	Accepted orders go to the **sequencer**, which stamps each with a strictly increasing sequence ID and writes it to the **event log**: a single, deterministic order of events.
4	The **matching engine** consumes events in order. One **single-threaded** order book per symbol matches by **price-time priority**, with no locks and fully in memory.
5	Every event is replicated to a **hot standby**. The standby has the identical state, so failover takes seconds without losing orders.
6	Executions and book changes flow to the **market data publisher**, which sends L1 / L2 data and candlesticks to subscribers (multicast, with sequence numbers).
7	The **reporter** sends execution reports to brokers and writes the audit trail for settlement and regulators.

6. Deep dive

Order book

- Per symbol: **buy side (bids)** sorted by price descending, **sell side (asks)** by price ascending; at each **price level** a FIFO queue of orders (time priority). **Price-time priority (FIFO matching):** best price first, then earliest.
- Data structures: price levels in a map/tree (or array indexed by price tick for O(1)), each level a doubly linked list; plus a hash map `order_id -> order node` for O(1) cancel. Operations: place O(1), cancel O(1), match O(1) per fill. Best bid/ask pointers cached.
- Matching example: incoming buy limit 100 @ 10.05 sweeps the best asks <= 10.05 in order; remainder rests in the book.

Matching engine and sequencer

- The **sequencer** assigns a strictly increasing sequence number to every inbound and outbound event and writes it to the **event store** (memory-mapped file). This gives **determinism**: replaying the same event stream produces the same book and trades (event sourcing), ensuring fairness and easy recovery/audit.
- The matching engine for a symbol is **single-threaded** (no locks, no context switches, CPU-pinned, cache-friendly, no GC; C++/Rust/Java with garbage-free techniques) and processes events in sequence order. Parallelism comes from **sharding by symbol** across cores/servers, not within a symbol.

Low latency techniques

- Everything on the critical path stays on **one server** to avoid network hops: gateway -> order manager -> sequencer -> engine can be co-located and communicate via **shared memory / mmap ring buffers (LMAX Disruptor / Aeron)**, **no database** on the hot path, **kernel bypass** networking (DPDK/RDMA), busy-spin threads, NUMA/CPU pinning, avoid logging and allocation.
- Reduce serialisation overhead with compact binary protocols (SBE) instead of JSON/FIX text internally.

High availability: hot-warm-cold

- **Hot standby:** the primary sequencer/engine replicates every event (before ack) to a secondary via Raft-like consensus or synchronous replication; secondary applies events and holds an identical book. On primary failure, **failover within seconds** via election (leader based on heartbeats); clients reconnect and replay from last seq.
- Event store flushed to disk and replicated; periodic **snapshots** (order book state) cut recovery time; cold backup in another site for disaster recovery (RPO ~ 0, RTO seconds).

Risk and validation (pre-trade)

- Order manager performs **risk checks** (daily volume, symbol allowed, funds/margin, price collars, rate limits) and maintains order state; **wallet/funds** check against the client's account to avoid selling what they do not own.

Market data publishing

- Engine emits execution events; market data publisher builds **L1** (best bid/ask), **L2** (aggregated levels), **L3** (per order) and **candlestick** charts, and distributes via multicast / pub-sub so all subscribers get data fairly at the same time; retail gets delayed or snapshot feeds. Archive history to a time-series DB for reporting.

Fairness and auditability

- Deterministic sequencing, precise timestamps (PTP/GPS clock sync), equal-length network cables and **multicast** for fan-out ensure no participant gets an advantage. Immutable event logs allow regulators to reconstruct any moment.

Network and security

- Public services (market data, retail UI) separated from the **private** trading network; DDoS protection at the edge (rate limits, allow lists, connection caps); TLS/mTLS; strict authentication of members.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Threading	Multi-threaded engine with locks / single-threaded per symbol	Single-threaded per symbol: deterministic, no lock contention, faster.
Messaging	Network queue (Kafka) / shared-memory ring buffer	mmap/Disruptor on the hot path; Kafka only for off-path consumers.
Replication	Async (fast, might lose) / synchronous consensus	Synchronous replication to hot standby: no lost orders.
State recovery	Replay from start / snapshot + tail	Snapshots + event tail for seconds-level recovery.

8. Scaling, reliability, failure modes

- Shard symbols across matching engine instances; hot symbols get dedicated cores/servers.
- Vertical optimisation first (latency), then horizontal by symbol.
- Capacity at open/close auctions: over-provision, backpressure and circuit breakers (price bands).

9. Likely follow-up questions

- How do you ensure orders for the same symbol are processed in the exact order received?
- How do you recover after the matching engine crashes mid-trade?
- How would you implement auctions (opening/closing) and circuit breakers?
- How do you distribute market data fairly?

10. Common mistakes

- Putting a database or Kafka on the matching hot path.
- Multi-threaded matching on one order book.
- Ignoring deterministic replay and fairness.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: limit and market orders, cancel, order book and real-time market data, for about 100 symbols and 1,000 clients. The key requirements are ultra-low latency in the tens of microseconds, high availability, fairness (price-time priority) and auditability. A rough target is 100 K orders per second, with no lost or duplicate orders."
2. High-level design	10 min	"Orders come through a gateway to an order manager for risk checks, then a sequencer that assigns a global order and stores an event log. A single-threaded matching engine consumes events and updates an in-memory order book per symbol. Outputs go to a market data publisher and a reporter. A hot standby replicates the events."
3. Deep dive	25 min	"The key trick is determinism: a single sequencer order and a single thread per symbol means no locks and a reproducible result. The order book is a sorted set of price levels with FIFO queues, with O(1) cancel using a hash map to nodes. To reach microseconds I keep everything in memory, avoid GC and the network where possible, co-locate, use ring buffers like the Disruptor pattern and use shared memory or mmap for the event store. For HA I replicate the event stream to hot standbys, and use consensus or leader election for failover."
4. Wrap up	4 min	"Extras: market data via multicast with gap recovery, circuit breakers, auction opening and closing, regulatory audit logs, and how to scale to more symbols by partitioning symbols across matching engines."

12. 60-second recap / practice checklist

- Gateway -> order manager (risk) -> sequencer (event log) -> single-threaded matching engine -> market data + reporter.
- Price-time priority order book with O(1) operations; event sourcing for deterministic recovery.
- Microsecond latency by co-location, shared memory, no DB on the path; HA via hot standby + snapshots.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• LMAX Disruptor
• Martin Fowler: The LMAX architecture
• Raft consensus